# Betting Project Report

Ver-0.9

1. Project Definition

- Goal
  Develop a high-performance Football Bet Predictor (FBP) app.

- Data source: https://www.oddsportal.com/
  - Data elements: Football matches
    Date of match,
  - Home team name,
  - Away team name,
  - League name,
  - Odds, for the following 12 Queries:
    Home Team Win,
    eg: 2.5, meaning if you bet on Home Team Win query, and if it actually happens (if you win the bet), then you will be returned 2.5 times your bet amount. 1.5 times of your bet amount will be your profit. If you lose the bet, then you will lose the bet amount.
    Game Draws
    Away Team Win
    Both Team Score
    Not Both Team Score
    Score Over 1.5
    Score Under 1.5
    Score Over 2.5
    Score Under 2.5
    Score 1:0
    Score 2:1
    Score 1:1
  - Actual Scores, eg: 2:1

- Book maker of choice: Bet365

2. Project Achievements

- Prediction is monthly profitable for selective queries.

  The following chart shows a result of test. A model trained by Normal Process is tested on a test dataset of 734 football games. The graph shows how the profit develops as we stake on the games according to the model's recommendations.

The chart reads as follows:

- The focus query is Away Team Win.
- The average profit per game is USD 0.55.
- The stability, or volatility, of profit is depicted.
- Model's answer "Away Team Win" is right at the probability of 73%.
- Model answer is "Away Team Win" on 29% of total "Away Team Win" events.

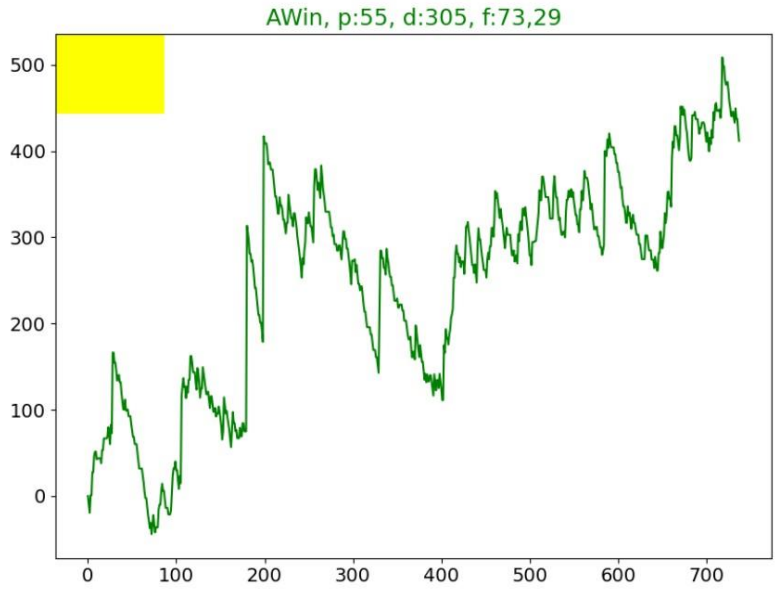



Fig-01. Betting profit progress over games, while **testing** a prediction model, on the Away Team Win query. Horizontal axis is for games to stake on, while the vertical is for profit accumulated.

Behind the chart, we have the following numbers:

- The average stake per game is USD 3.5 (or any currency).
- Each month, we have over 300 games to bet on.
- Safety is 10X - We keep a capital of 10 times the average stake per game, in case we lose 10 times in a row.

From the chart, we can deduce the following calculations:

Average profit rate per game
= Average profit per game / Total capital working
= Average profit per game / (Average stake per game * Safety)
= USD 0.55 / USD 3.5 / 10 = 0.0157

Compound profit rate per month, which is when we add profit to the capital,
= [# Games to stake a month]-th power of [ 1 + Average profit rate per game]
= $300^{th}$ power of (1 + 0.0157) = 107

The normal profit rate per month
= [Average profit rate per game] x [# Games to stake a month]
= 0.0157 x 300 = 4.71

For the Safety of 200X, we have

- Compound profit rate = 1.2
- Normal profit rate per month = 0.23

For the Safety of 100X, we have

- Compound profit rate = 1.6
- Normal profit rate per month = 0.47

For the Safety of 10X, we have

- Compound profit rate = 107
- Normal profit rate per month = 4.7

The chart shows that we need minimum Safety of 100X.
To reduce the safety requirements, we have to straighten, or stabilize, the graph. In other words, we have to reduce the volatility of profit graph.

I did everything to reduce the volatility. I even put the volatility directly in the loss function, which the model aims to reduce during training, but I vain.

- Our Exhaustive DNN Convergence seems to work. Here is an evident.
  - As soon as I improved the Normal-Process with my Exhaustive DNN Convergence algorithm, there began to appear some models showing steady profit rise in training phase. Before the Exhaustive, it was almost impossible to see a model showing profit rise in training phase, let alone the validation phase, even more let alone the training phase.
  - Different models produced by the Normal-Process boosted with Exhaustive DNN Convergence algorithm would show the same, systematic, tendency.

I don’t think everybody working on a model-building project would experience such an adversary, anti-intuition phenomenon that a betting model-building project with poor dataset generates. According to human intuition who lives in this planet, and like all regular model-building processes believe, if a model is good in training phase, and if the model is lucky to be also good in validation phase, then the model should almost also be good, though less, in test phase. This is our intuition formed from our experience while interacting with the nature and society, reflecting the nature.

Betting project seemed to be against this intuition. Even if a model holds to be good in all the training, validation, and testing, we have a doubt that the model was good accidently on the given dataset – training dataset, validation dataset, and testing dataset. If we change the dataset, the model might suddenly change to be not good at all. That acts as a reason that we have to rebuild a model if the underlying dataset or its constitution transit, or change significantly, to a new world. This is consistent with that a model is just a summary of underlying dataset.

So, we can’t completely trust the model, even if it is good at a specific dataset. But what if we have very limited number of example data in the dataset. (Of course, critics would say that this kind of extremity may lead to even doubting the security block-chain. Hash function, which is the founding stone to block-chain, can’t give a unique hash value, actually. Two different data CAN give the same hash value. Block-chain believes if two data give the same hash value, then they are the same. Yes, almost the same. The probability they are not the same is almost zero but not zero. The probability that a block-chain adopts a fake block is almost zero but not zero. The probability that a model that was good in training and validation is not good in testing and production is significantly not zero. More so in betting with poor underlying dataset. A poor dataset can’t represent the actual population.)

There is some evident that encourages us to trust the model-building process, thus the models. Different models produced by the process tend to give almost identical results, as show below:

If Fig-02, all the 18 different models on query “Away Team Win” by and large have almost the same big rise/fall games, where the profit goes up/down, respectively. This may mean that the rise/fall games are invariant in the process (function). If the process has systematic invariants, we can further refine the process for a better one. Our Meta-Process will be soon ready for this refinement. **Meta-Process is designed to reduce the fall and fall games.** By reducing the falls, we can have higher profitability, even if the rise and rise games are contaminated a little during the process.

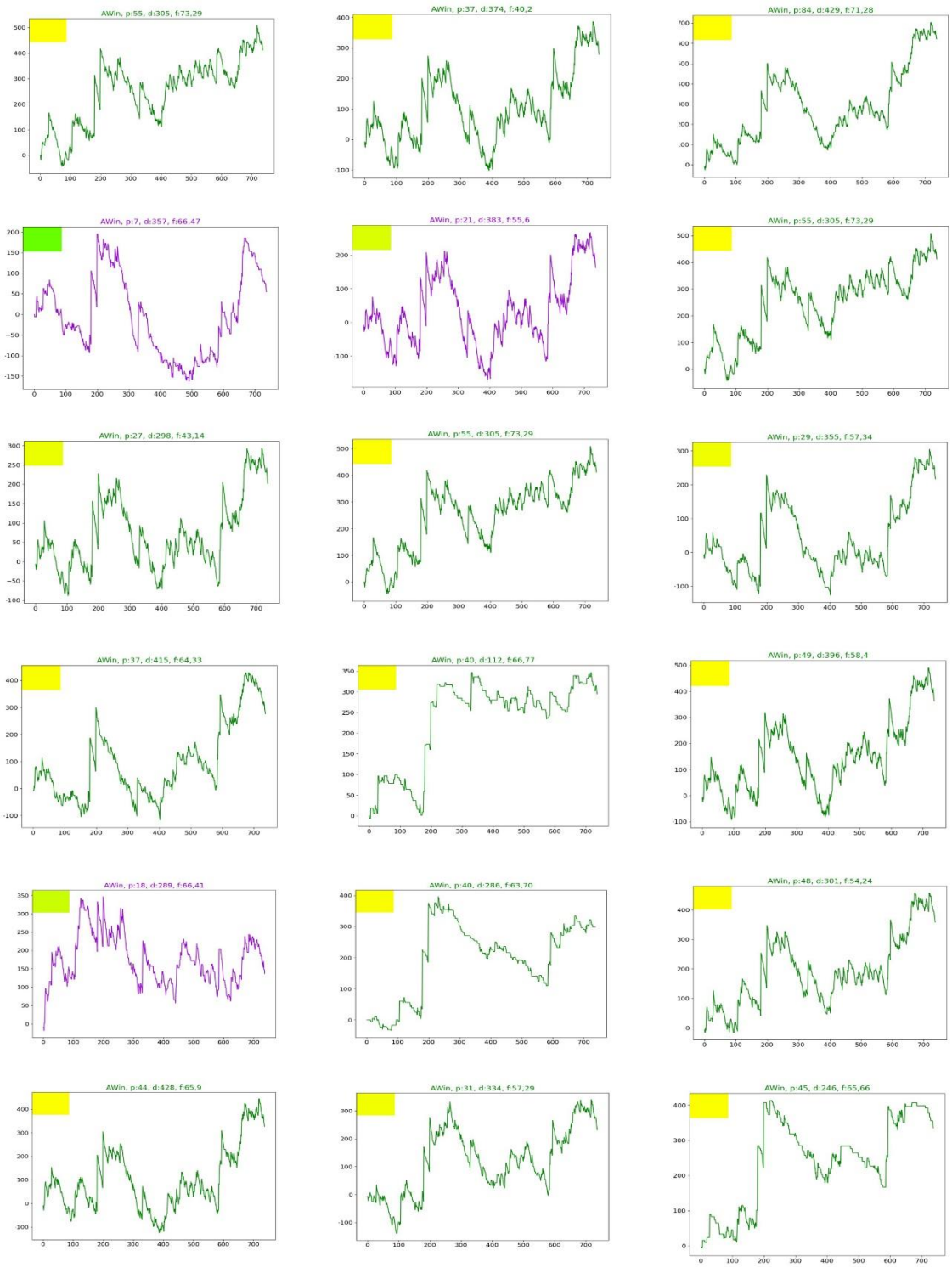



Fig-02. Betting profit progress over games, while **<u>testing</u>** 18 prediction models, **all on the Away Team Win query**. Horizontal axis is for games to stake on, while the vertical is for profit accumulated. <u>All of them follows an identical systematic tendency by and large, encouraging to trust the model-building process.</u>

3. Future Roadmap

- We are going to stabilize the profit graph, by using our Meta-Process.

  The profit graph seems to be monthly profitable, meaning we will have positive profit in every monthly time interval, at a high probability. Positive profitability seems to be guaranteed for any monthly interval, but not for some weekly interval or daily interval.
  We need to achieve weekly profitable prediction, or even daily profitable prediction.

  Further stability will be achieved by our Meta-Process. Meta-Process can be recursively applied to the output model until we are satisfied.

- We are going to save more queries.

  We now fail to find profitable prediction models on most of queries (i.e. we lose most queries), as shown in the following charts:

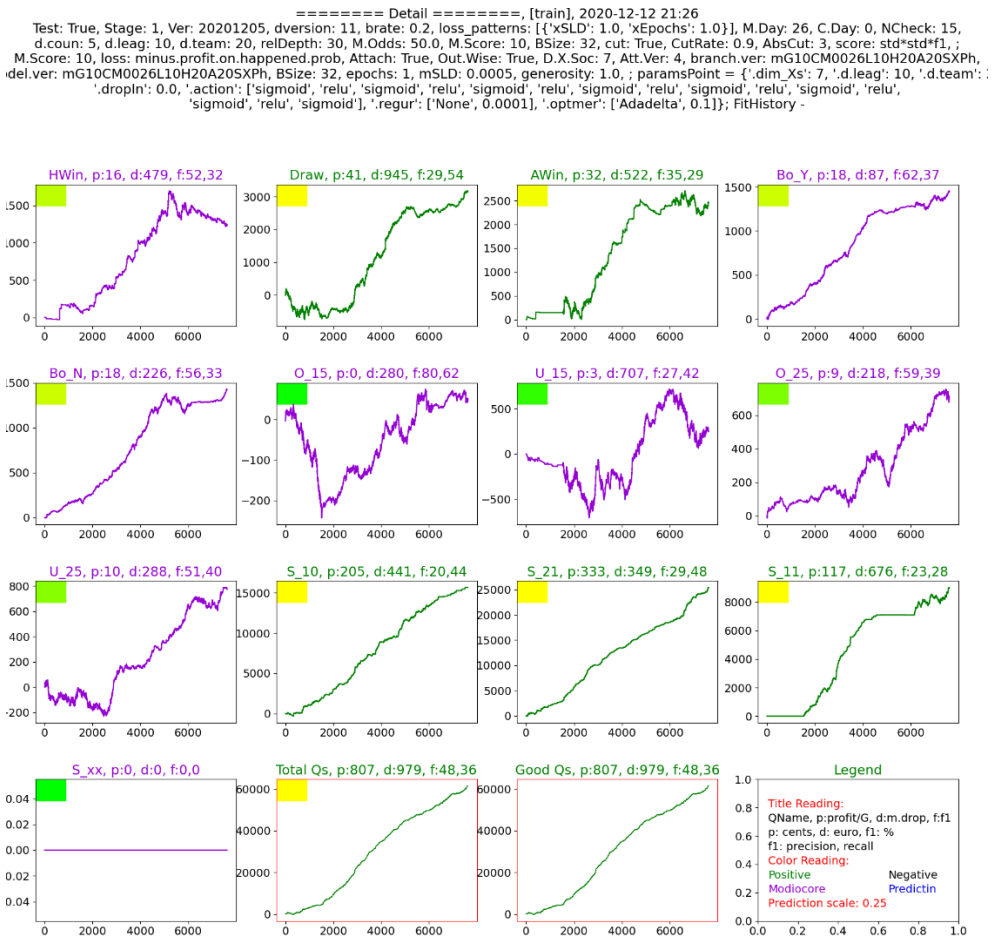



Fig-03. Betting profit progress over games, while **<u>training</u>** prediction models, on each of the 12 queries. Horizontal axis is for games to stake on, while the vertical is for profit accumulated.

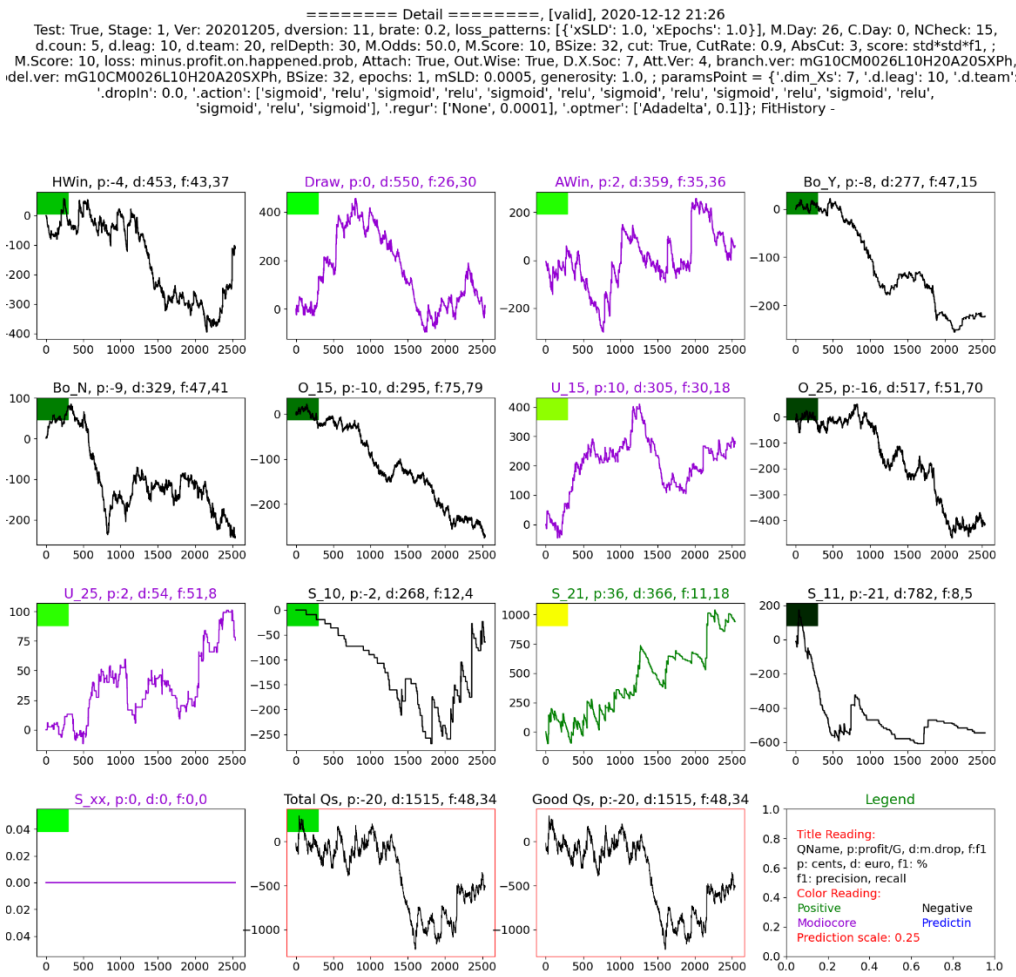



Fig-04. Betting profit progress over games, while **validating** prediction models, on each of the 12 queries. Horizontal axis is for games to stake on, while the vertical is for profit accumulated.

From Fig-03, we can see that we can produce quite good models on training phase. But from Fig-04, we see that those models are not passable on validation.

If a model shows different quality between training and validation, then the model has no systematic perception/knowledge about the data, and we don't need even test the model.

Most models are good in training phase, but not good in validation phase, meaning that the models have perceived too much something wrong about the training data that don't exist in validation data, so couldn't show good profit in validation and couldn't pass validation.

Once a model is good in both training and validation, then it can be passed on to test phase, where it will be tested on a new data that the model hasn't seen

during training/validation. If the model is still good in test phase, then we have not reason to drop the model.

The model shown in Fig-01, which is a test result, was good in both training and validation.

Because the dataset we are using for this project is extremely poor, it is almost impossible to find a passable model if we rely on regular techniques. I have proven it for 10 months with desperate trial-and-errors. The difficulty led me to seek innovative solutions. Most of the solutions also failed to give a yield a good model.

The more models we save and use as a betting resource, the more profit we can expect. It will have the effect of having more games in a month, because if we save 3 queries and 3 corresponding good models, then we will have 3 times 300 games per month = 900 games to stake on. That big number of games will reduce the entropy of profit, and we will be statistically safe. The strategy should be “bet small on lots of games” rather than “bet much on a very likely-profitable game”. The cost of orders in betting market is flat. The is a monthly fee, not per-order fee, in using betting API, which is different from per-order fee in trading.

- We want to improve the data.

Our data is collected by scraping web sites, by manually correcting the scrapped data. Web sites would use multiple names for a single team, for example.

We found that some institutional competitors have very rich commercial football game data, over 40,000 game records each with 100 columns, while our dataset has only 12,000 imperfect game records each with only 20 columns.

A DL model is nothing but a well-extracted summary of data. No model can be better than the underlying data. Our processes of building model have shown an excellent performance on a rich dataset.

4. Comments on OZTG prediction projects

The Project ABC and Project DE is not likely to suffer poor dataset as much as the betting project did. Project DE, in particular, has as much data as our competitors. And the data is well-formed and richly time-granulate, if the data come from liquid coins, like BTC and ETH.

The projects can benefit from a wide range of additional data, like Social Media Feeds.